

Zwischenbericht - Campus Next-Gen Data-Hub

Contents

Zwischenbericht — Campus Next-Gen Data-Hub	1
1. Projektziel & Kontext	1
2. Erreichte Meilensteine (Stand 06.06.2026)	2
3. Architektur (Kurzüberblick)	3
4. Besonderheit Datenqualität der Quell-CSVs	3
5. Probleme & Lösungen	3
6. Offene Punkte / Fragen an die Betreuer	4
7. Anforderungs- und Szenarien-Status	5
8. Nächste Schritte	5
Anhang: Repository-Struktur	5
Anhang: Anforderungen & Umsetzungsstand	6
Anforderungen & Umsetzungsstand	6
1. System-Anforderungen (aus dem Kickoff)	6
2. Szenarien-Status	7
3. Offene Punkte / Fragen an die Betreuer	7
4. Bereits umgesetzte Airbyte-Objekte (Stand 2026-06-06)	8

Zwischenbericht — Campus Next-Gen Data-Hub

Evaluation von Airbyte als ETL-/Integrationswerkzeug (Ablösung von Talend)

Modul	INF-M Modul Projekte SoSe '26
Gruppe	Airbyte
Bearbeiter	Bräutigam Rebecca rbraeuti@stud.hs-offenburg.de Horst Isabella ihorst@stud.hs-offenburg.de Lahres Timo tlahres@stud.hs-offenburg.de
Stand	06.06.2026
Abgabe	07.06.2026 (Zwischenbericht + Doku)

1. Projektziel & Kontext

Im Projekt evaluieren wir [Airbyte](#) als modernes ETL-/Datenintegrationswerkzeug mit dem Ziel, das bisher eingesetzte **Talend** in der Hochschul-IT abzulösen. Die gesamte Evaluationsumgebung läuft **lokal in Docker Desktop** und bildet einen realistischen Ausschnitt der Hochschul-Datenlandschaft (anonymisierte Studierenden-, Gebäude-, Instituts- und Personaldaten) ab.

Die Evaluation ist in **sechs Testszenarien** gegliedert (DB-Connector, Facility-Management-Sync, Bild-BLOBs, Studenten/Personal-Mapping, Incremental Sync/IdM, Web-APIs) — siehe [testszenarien.md](#).

2. Erreichte Meilensteine (Stand 06.06.2026)

Meilenstein (lt. Betreuer-Mail)	Status	Beleg / Doku
Installation des Systems	[OK] DB-Stack + Airbyte (abctl) lauffähig	installation-guide.md
Zugang für Betreuer	[teilw.] dokumentiert, Einrichtung im Termin	zugang.md
Einfacher ETL-Prozess	[OK] durchgeführt & verifiziert (Postgres→Postgres)	etl-prozess.md
Beginn der Dokumentation	[OK] README + 8 Dokumente unter docs/	dieses Repo

2.1 Installation

Das Setup ist vollständig skriptbasiert und reproduzierbar:

- `scripts/install.ps1` (Windows) bzw. `scripts/install.sh` (Linux/macOS) startet den kompletten Datenbank-Stack (Source-PostgreSQL, Ziel-PostgreSQL, Ziel-MySQL, CSV-File-Server), wartet auf den `healthy`-Status und lädt die Testdaten.
- `scripts/setup-airbyte.ps1` / `scripts/setup-airbyte.sh` installiert Airbyte Community Edition über das offizielle CLI `abctl` in einem lokalen Kubernetes-Cluster (`kind`) innerhalb von Docker Desktop.
- **Plattformen:** Das Setup steht für **Windows, Linux und macOS** bereit (identische Logik, plattformspezifische Skripte).

Alle vier Datenbank-/Server-Container laufen verifiziert im Zustand `healthy`.

2.2 Datenbasis (Source-PostgreSQL)

Die anonymisierten Testdaten sind geladen:

Tabelle	Zeilen	Lademechanismus
<code>fm_gebaeude</code>	25	<code>scripts/load_fm_gebaeude.py</code>
<code>fm_inst</code>	2.083	<code>scripts/load_fm_inst.py</code>
<code>k_plz</code>	34.172	<code>scripts/load_k_plz.py</code>
<code>fm_rna</code>	379	<code>scripts/load_json.py</code>
<code>hso_personal</code>	870	<code>scripts/load_json.py</code>
<code>hso_students</code>	0	[!] nur via File-Connector (s. Kap. 4)
<code>fm_stamm</code>	0	keine Quelldatei (s. Kap. 5)

2.3 Angelegte Airbyte-Connectoren & erster ETL-Lauf

In Airbyte sind angelegt und per Verbindungstest grün:

- **Sources (5):** HSO Source PostgreSQL (Postgres, Update-Methode *User Defined Cursor*), sowie vier File-Connectoren (`local`, `/local/*.csv`): HSO CSV `hso_students`, HSO CSV `k_plz`, HSO CSV `fm_gebaeude`, HSO CSV `fm_inst`.
- **Destinations (2):** HSO Dest PostgreSQL (Port 5434), HSO Dest MySQL (Port 3306, SSL aus, `allowPublicKeyRetrieval=true`, `Raw-DB destdb`).

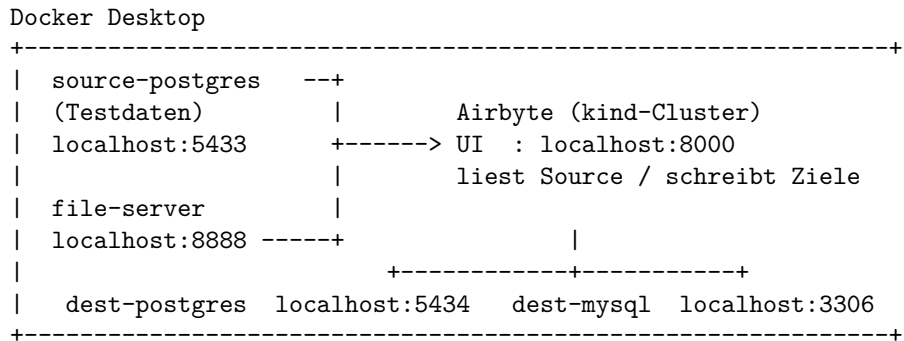
Der **erste ETL-Prozess** (Connection HSO Source PostgreSQL → HSO Dest PostgreSQL, Streams `fm_gebaeude` + `k_plz`, Modus *Full Refresh / Overwrite*) wurde erfolgreich ausgeführt. Verifikation in der Ziel-DB:

Stream	Zeilen Source	Zeilen Ziel (destdb)
<code>fm_gebaeude</code>	25	25
<code>k_plz</code>	34.172	34.172

Damit sind sowohl der **DB-Connector** als auch der **File-Connector** (Szenario 1) nachweislich lauffähig.

3. Architektur (Kurzüberblick)

Detaillierte Beschreibung in [architektur.md](#). Kern: alle Dienste laufen in Docker Desktop in einem gemeinsamen Netzwerk (`airbyte_net`). Airbyte selbst läuft in einem kind-Cluster und erreicht die Datenbanken über `host.docker.internal`.



4. Besonderheit Datenqualität der Quell-CSVs

Die bereitgestellten CSV-Dateien ließen sich **nicht** per direktem PostgreSQL-COPY laden (Details in Kap. 5). Wir haben daher tolerante Python-Loader implementiert, die die Daten nach dem Containerstart bereinigt einspielen. Eine Datei — `hso_students.csv` — ist strukturell so inkonsistent, dass sie aktuell nicht zuverlässig in die relationale Source-DB geladen werden kann; Studierendendaten werden stattdessen über den **Airbyte File-Connector** als Flatfile-Quelle eingebunden.

5. Probleme & Lösungen

Dieses Kapitel ist gemäß Betreuer-Mail explizit gefordert.

5.1 Windows-/Umgebungsspezifische Hürden

Problem	Ursache	Lösung
<code>install.ps1</code> meldete „Python gefunden“, JSON-Laden schlug aber fehl	<code>python</code> ist unter Windows oft nur der Microsoft-Store-Platzhalter ; <code>Get-Command</code> findet ihn, er liefert aber keine echte Version	Echte Versionsprüfung (<code>py/python/python3</code>); zusätzlich Docker-Fallback , der die Daten ganz ohne Host-Python lädt
<code>setup-airbyte.ps1</code> fand kein <code>abctl</code> -Asset	Falsches Namensschema (<code>abctl_Windows_amd64.zip</code>) — korrekt ist <code>abctl-<version>-windows-<arch>.zip</code> — zudem liegt <code>abctl.exe</code> in einem Unterordner	Asset per Muster ermitteln, aus Unterordner entpacken; Architektur-Erkennung <code>PowerShell-7-fest</code>
File-Server-Container dauerhaft unhealthy , Setup lief in Timeout	Healthcheck nutzte <code>localhost</code> → im Container zuerst IPv6 (<code>::1</code>), <code>nginx</code> lauscht aber nur auf IPv4	Healthcheck auf <code>127.0.0.1</code> umgestellt

5.2 Datenqualität der Quell-CSVs (Hauptproblem)

Der SQL-Init lud die drei Kern-Tabellen **gar nicht** — COPY brach unter `ON_ERROR_STOP` bereits an der ersten fehlerhaften Datei ab, wodurch alle folgenden leer blieben. Die konkreten Defekte:

- **fm_gebaeude.csv**: unquotierte Kommas in Textfeldern (z. B. „Hörsäle,Bib.,RZ“), eingebettete Header-Zeilen.
- **k_plz.csv**: **3.417 Header-Zeilen** mitten in den Daten (zusammengesetzte Einzel-Exporte), Zeilen mit zu wenig Feldern, ein Feld (`krskfz`) breiter als das Schema (Kreis-Namen).
- **fm_inst.csv**: 86 Spalten (Tabelle nutzt 24), semikolon-getrennt, vereinzelt **NUL-Bytes**, doppelt-kodierte UTF-8-Umlaute.
- **hso_students.csv**: **strukturell defekt** — Datenzeilen haben mehr Spalten (~50–56) als der eigene Header (40), dazu unbalanciertes Quoting und Float-formatierte Integer.

Lösung: tolerante Python-Loader (`scripts/load_*.py`), die Header filtern, fehlerhafte Zeilen reparieren bzw. überspringen, NUL-Bytes und Mojibake bereinigen und Typkonvertierungen best-effort vornehmen. Der SQL-COPY-Schritt wurde entfernt (`01_load_data.sql`).

5.3 Airbyte/abctl-spezifische Hürden

Airbyte Community läuft über `abctl` in einem **kind-Kubernetes-Cluster**. Daraus ergaben sich mehrere nicht-offensichtliche, in der offiziellen Doku nicht beschriebene Hürden, die wir analysiert und gelöst haben:

Problem	Ursache	Lösung
File-Connector (<code>local</code>) findet <code>/local/*.csv</code> nicht	Connector-Pods (<code>kind</code>) sehen das Docker-Volume <code>oss_local_root</code> nicht	CSV-Verzeichnis beim Install via <code>abctl local install --volume "...:/local"</code> direkt in den Cluster mounten
<code>--volume</code> mit Windows-Pfad: <code>is not a valid volume spec</code>	<code>abctl</code> trennt den Volume-String stur an <code>:</code> , der Laufwerks-Doppelpunkt (<code>C:</code>) kollidiert	Pfad in MSYS-Form <code>/c/Users/...</code> angeben
Alle Connector-Tests/Syncs hängen Pending	Aktiviertes lokales Volume erwartet PVC <code>airbyte-local-pvc</code> in jedem Job-Pod; es existierte nicht (<code>persistentvolumeclaim not found</code>)	PV (<code>hostPath /local</code>) + PVC <code>airbyte-local-pvc</code> anlegen; <code>JOB_KUBE_LOCAL_VOLUME_ENABLED=true</code> + Neustart von <code>launcher/worker</code>
<code>abctl local credentials --email ... --password ...</code> schlägt fehl (<code>unable to determine organization email, invalid character '<'</code>)	Kombinierter Aufruf löst einen Org-Lookup aus, der HTML statt JSON liefert	E-Mail und Passwort in zwei getrennten Aufrufen setzen (erst <code>--email</code> , dann <code>--password</code>)
Connector-Auswahl heißt „ Postgres “ (nicht „PostgreSQL“); Default-Update-Methode ist CDC	—	In der UI „Postgres“ wählen; Update-Methode auf <i>User Defined Cursor</i> stellen (CDC bräuchte <code>wal_level=logical</code>)

Alle Lösungen sind in `scripts/setup-airbyte.ps1/.sh` automatisiert und in [airbyte-setup.md](#) / [etl-prozess.md](#) dokumentiert.

6. Offene Punkte / Fragen an die Betreuer

Ebenfalls gemäß Betreuer-Mail gefordert.

1. **hso_students.csv** — **Soll-Struktur?** Die Datei ist nicht eindeutig ladbar (mehr Datenspalten als Header, defektes Quoting). Können Sie eine korrigierte Datei bereitstellen oder die beabsichtigte Spaltenstruktur/das Export-Format nennen? Dies blockiert aktuell Szenario 4 (Account-Mapping in die Source-DB).

2. **fm_stamm** (Raumstammdaten): Es liegt keine Quelldatei vor. Wird diese Tabelle benötigt, und woher sollen die Daten kommen?
 3. **Zugang für Betreuer:** Airbyte Community Edition ist im Wesentlichen Single-User und läuft auf `localhost`. Wie soll Ihr Zugang erfolgen — gemeinsamer Admin-Login während des Vor-Ort-Termins, oder ist ein Remote-Zugang (z. B. Tunnel/VM) gewünscht? (Vorschlag in [zugang.md](#).)
 4. **Sync-Strategie:** Wir nutzen den Cursor-Modus (`updatedat`) statt CDC, da CDC zusätzliche PostgreSQL-Konfiguration (logical WAL, Replication Slot) erfordert. Ist CDC für die Evaluation gewünscht?
 5. **Scope:** Welche der sechs Szenarien haben für die Bewertung Priorität?
-

7. Anforderungs- und Szenarien-Status

Eine vollständige Gegenüberstellung aller Kickoff-Anforderungen und der sechs Szenarien mit Bewertung der Airbyte-Eignung und unserem Umsetzungsstand steht in [anforderungen.md](#). Kernbefunde:

- **Erfüllt:** Open Source/Community, PostgreSQL- & MySQL-Anbindung, CSV/JSON/Excel-Dateien, Logging/Monitoring, einfacher ETL-Lauf (verifiziert).
- **Einschränkungen ggü. Talend:** kein OSS-Connector für Informix; kein XML nativ; keine freie Code-Snippet-Ausführung (Mapping nur via dbt-SQL/Custom-Connector). Diese Punkte sind die zentralen Evaluationsbefunde.

8. Nächste Schritte

- Szenario 1 abrunden: Sync auch nach MySQL + ein File→DB-Sync (Nachweis File-Connector-Last).
 - Weitere Szenarien: FM-Denormalisierung (dbt/View), BLOB-Bilder, Incremental Sync (IdM), Web-APIs (PostgREST/SOAP).
 - Klärung der offenen Fragen aus Kap. 6.
-

Anhang: Repository-Struktur

Siehe [README.md](#). Setup in unter 20 Minuten via `scripts/install.ps1` + `scripts/setup-airbyte.ps1`.

Anhang: Anforderungen & Umsetzungsstand

Anforderungen & Umsetzungsstand

Diese Übersicht fasst alle Anforderungen aus dem Kickoff ([moodle/Kickoff.md](#)) und den sechs Szenarien ([moodle/Projektszenarien.md](#), ausführlich in [testszenarien.md](#)) zusammen und bewertet, **was Airbyte kann** und **wie weit wir sind**.

Stand: 2026-06-06. Legende: [OK] erledigt · [teilw.] teilweise/in Arbeit · [offen] offen · [!] Einschränkung/Risiko.

1. System-Anforderungen (aus dem Kickoff)

#	Anforderung	Airbyte-Fähigkeit	Status im Projekt
A1	Open Source + aktive Community	Airbyte OSS (MIT/ELv2), sehr aktive Community	[OK] erfüllt (Auswahlkriterium)
A2	DB-Anbindungen (min. Informix, MySQL, PostgreSQL)	Postgres + MySQL als Source/Destination vorhanden; Informix nur als Enterprise/Db2-nah, kein OSS-Connector	[teilw.] PG+MySQL [OK], Informix [!] Lücke
A3	Datei-basiert: CSV, Excel, JSON, XML	File-Connector: CSV/JSON/Excel/Feather/Parquet/JSON/XML [OK]; XML nativ nicht	[teilw.] CSV/JSON/Excel [OK], XML [!]
A4	SOAP- und REST-APIs abfragen	REST: Connector-BUILDER (Low-Code) / HTTP; SOAP: nicht nativ, nur als HTTP-POST mit XML-Parsing	[offen] (Szenario 6)
A5	Daten-Mapping/Transformation über eigenen Code	Transformationen via dbt (SQL) bzw. Custom Connector; kein freies Code-Mapping pro Feld wie in Talend	[teilw.] Konzept steht, [!] Paradigmenwechsel zu dbt/SQL
A6	Low-Code REST-API bereitstellen	Airbyte stellt selbst keine Daten-API bereit → externer Dienst PostgREST	[offen] (Szenario 6a, PostgREST vorgesehen)
A7	Code-Snippets ausführen (Python/JS/Groovy/Selenium)	Airbyte führt keine freien Skripte aus; nur dbt-SQL oder eigener Connector (Python-CDK/Low-Code)	[!] Lücke ggü. Talend (zentraler Evaluationsbefund)
A8	Logging & Monitoring von Jobs	Job-Historie, Status-UI, Logs pro Sync/Attempt, Timeline	[OK] vorhanden
A9	Usability / einfache Konfiguration	Web-UI, Connector-Kataloge, geführte Setups	[OK] (mit abctl-spezifischen Stolpersteinen, s. installation-guide)
A10	Integration in die Hochschul-IT	Läuft lokal/on-prem via abctl (kind); API/Terraform-Anbindung möglich	[teilw.] lokal evaluiert; Produktiv-Integration offen

2. Szenarien-Status

Szenario	Inhalt	Airbyte-Eignung	Status	Nächster Schritt
1 Testdaten einspielen	Daten in MySQL und PostgreSQL; Postgres- + File-Connector testen	[OK] gut	[teilw.] Postgres-ETL live verifiziert (fm_gebaeude 25, k_plz 34.172 in destdb); alle 5 Sources + 2 Destinations angelegt , File-Connector-Tests grün	Sync auch nach MySQL + File→DB-Sync zeigen
2 Facility Management	PG-Tabellen inst/geb/stamm; 1 denormalisierte Raum-Tabelle in MySQL	[teilw.] Sync ok, Joins nur via dbt/View	[teilw.] fm_inst/fm_gebaeude geladen; fm_stamm ohne Quelle	View/dbt für fm_raeume, dann nach MySQL
3 Bilder als BLOB	>1000 Bilder per API → BYTEA/Blob in DB; später als Datei exportieren	[!] eingeschränkt (BYTEA-Handling)	[offen] Skripte scripts/images/*.py vorhanden, nicht ausgeführt	Bilder laden, BYTEA→MySQL-Sync prüfen
4 Mapping Studenten/Personal	Random-Daten, Account-Generator, in neue Tabellen schreiben	[teilw.] via dbt/Custom	[offen] scripts/mapping/generate_accounts.py vorhanden; blockiert durch defekte hso_students.csv	Soll-Struktur von Betreuerdaten.py
5 IdM-System	hso_personal+hso_students → hso_user (MySQL), Sync bei Änderung; Bild-Verknüpfung	[OK] gut (Incremental + Dedup)	[offen]	Cursor updatedat + PK setzen, Incremental-Connection
6 Web APIs	6a REST (insert/update), 6b SOAP HISinOne	[!] REST via PostgREST/Builder; SOAP komplex	[offen]	PostgREST-Dienst; SOAP-Zugang von Betreuern abwarten

3. Offene Punkte / Fragen an die Betreuer

(siehe auch [zwischenbericht.md](#), Kap. 6)

1. **hso_students.csv** strukturell defekt (mehr Datenspalten als Header) → blockiert Szenario 4. Korrigierte Datei / Soll-Struktur?
2. **fm_stamm** (Raumstammdaten): keine Quelldatei vorhanden — woher kommen die Daten?
3. **Informix**-Anbindung: kein OSS-Airbyte-Connector. Ist Informix zwingend, oder reicht PG/MySQL für die Evaluation?
4. **Code-Snippet-Ausführung** (A7) ist Airbytes größte Lücke ggü. Talend. Wie wichtig ist dieses Kriterium für die Bewertung?
5. **SOAP/HISinOne**-Zugang (Szenario 6b) wird testweise bereitgestellt — wann?
6. **Sync-Strategie** Cursor (**updatedat**) statt CDC — für die Evaluation ausreichend?

7. Szenario-Priorisierung für die Bewertung?

4. Bereits umgesetzte Airbyte-Objekte (Stand 2026-06-06)

Sources: HSO Source PostgreSQL (Postgres, User-Defined-Cursor) · HSO CSV hso_students · HSO CSV k_plz · HSO CSV fm_gebaeude · HSO CSV fm_inst (alle File/local, /local/*.csv). **Destinations:** HSO Dest PostgreSQL (5434) · HSO Dest MySQL (3306, SSL aus, allowPublicKeyRetrieval=true). **Connection:** HSO Source PostgreSQL → HSO Dest PostgreSQL, Full Refresh | Overwrite, **erfolgreich gesynct**.